

ASSIGNMENT-12.1

Name: B.Chandu

HT. No: 2303A51737

Batch: 08

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

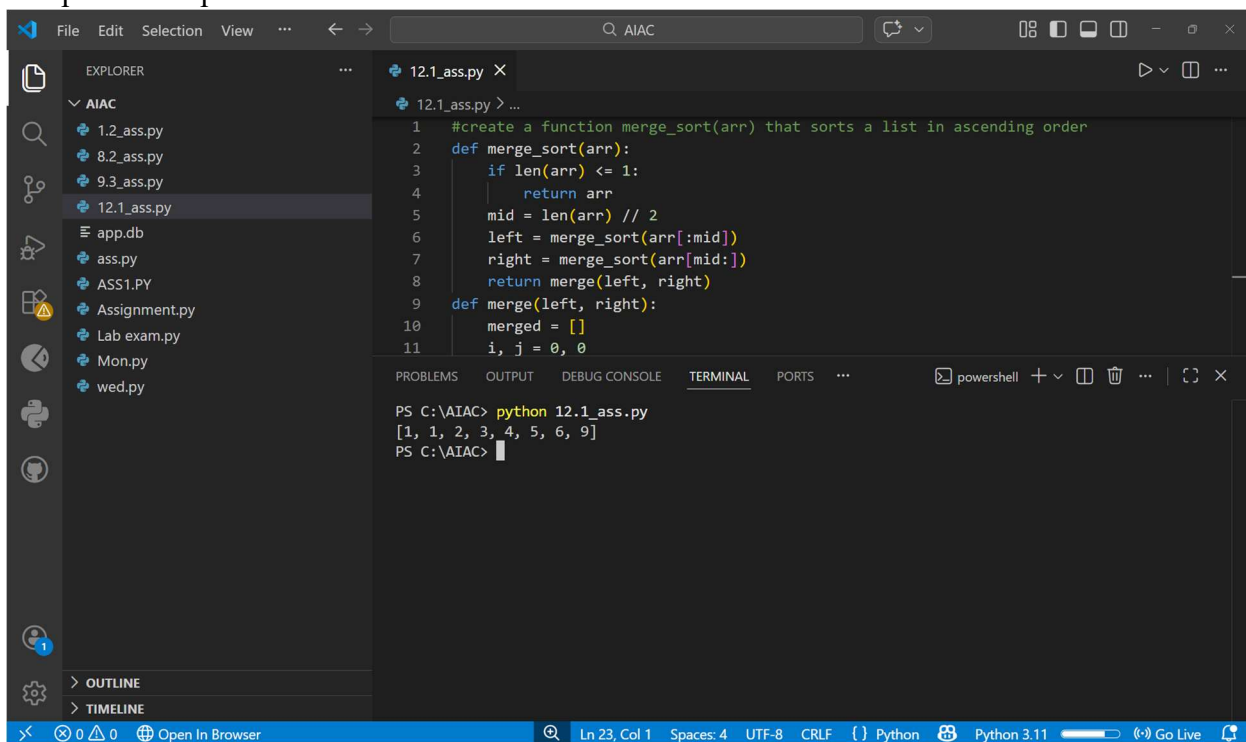
- Instructions:

- o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.

- o Ask AI to include time complexity and space complexity in the function docstring.

- o Verify the generated code with test cases.

- Expected Output:



The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'AIAC' with several files, including '12.1_ass.py'. The code editor displays the following Python code for a merge sort implementation:

```
1 #create a function merge_sort(arr) that sorts a list in ascending order
2 def merge_sort(arr):
3     if len(arr) <= 1:
4         return arr
5     mid = len(arr) // 2
6     left = merge_sort(arr[:mid])
7     right = merge_sort(arr[mid:])
8     return merge(left, right)
9
10 def merge(left, right):
11     merged = []
12     i, j = 0, 0
```

The terminal at the bottom shows the command `python 12.1_ass.py` being executed, resulting in the output `[1, 1, 2, 3, 4, 5, 6, 9]`.

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.

- Instructions:

- o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.

- o Include docstrings explaining best, average, and

worst-case complexities.

- o Test with various inputs.

- Expected Output:

```
23 #create a function binary_search(arr,target) returning the index of the target or -1 if not found
24 def binary_search(arr, target):
25     low = 0
26     high = len(arr) - 1
27     while low <= high:
28         mid = (low + high) // 2
29         if arr[mid] == target:
30             return mid
31         elif arr[mid] < target:
32             low = mid + 1
33         else:
34             high = mid - 1
35     return -1
36 print(binary_search([1, 2, 3, 4, 5, 6, 7, 8, 9], 5))
```

```
PS C:\AIAC> python 12.1_ass.py
[1, 1, 2, 3, 4, 5, 6, 9]
PS C:\AIAC> python 12.1_ass.py
4
PS C:\AIAC>
```

Task Description #3 (Real-Time Application – Inventory Management System)

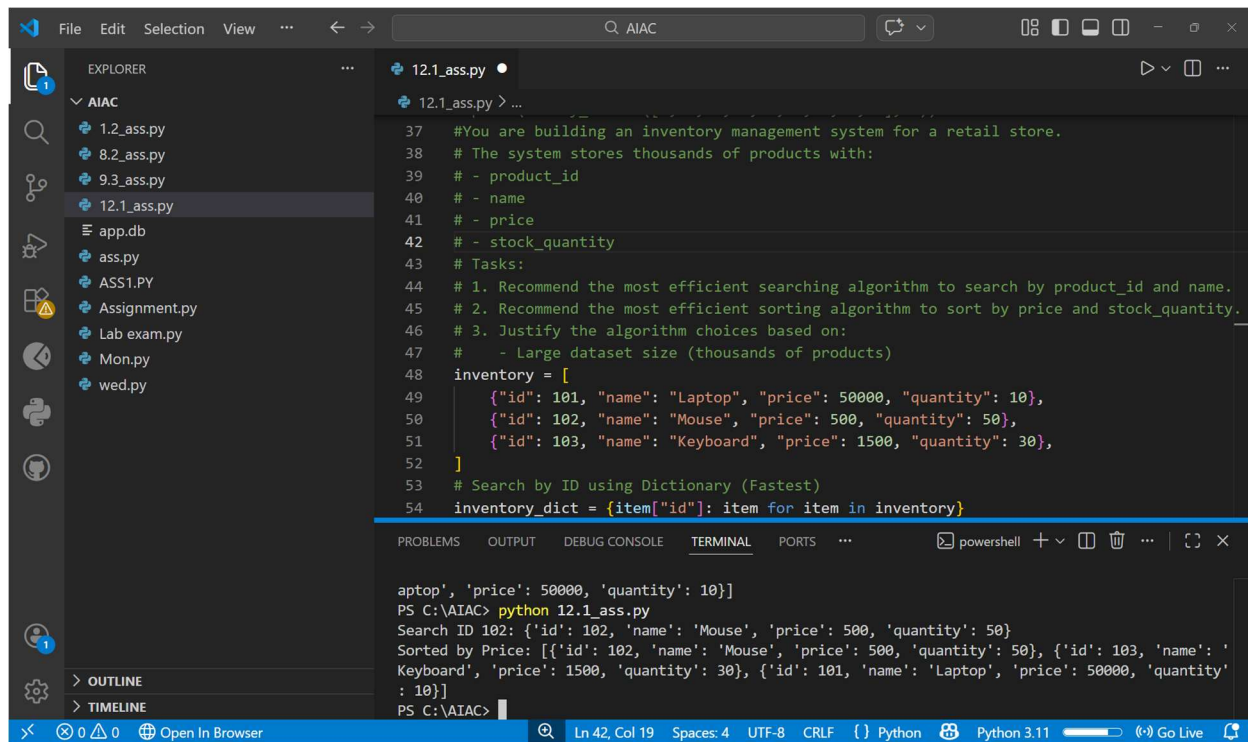
- Scenario: A retail store’s inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

- Task:

- o Use AI to suggest the most efficient search and sort algorithms for this use case.
- o Implement the recommended algorithms in Python.
- o Justify the choice based on dataset size, update frequency, and performance requirements.

- Expected Output:



```
File Edit Selection View ... Q AIAC
12.1_ass.py
12.1_ass.py > ...
37 #You are building an inventory management system for a retail store.
38 # The system stores thousands of products with:
39 # - product_id
40 # - name
41 # - price
42 # - stock_quantity
43 # Tasks:
44 # 1. Recommend the most efficient searching algorithm to search by product_id and name.
45 # 2. Recommend the most efficient sorting algorithm to sort by price and stock_quantity.
46 # 3. Justify the algorithm choices based on:
47 # - Large dataset size (thousands of products)
48 inventory = [
49     {"id": 101, "name": "Laptop", "price": 50000, "quantity": 10},
50     {"id": 102, "name": "Mouse", "price": 500, "quantity": 50},
51     {"id": 103, "name": "Keyboard", "price": 1500, "quantity": 30},
52 ]
53 # Search by ID using Dictionary (Fastest)
54 inventory_dict = {item["id"]: item for item in inventory}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ... powershell + v
aptop', 'price': 50000, 'quantity': 10}]
PS C:\AIAC> python 12.1_ass.py
Search ID 102: {'id': 102, 'name': 'Mouse', 'price': 500, 'quantity': 50}
Sorted by Price: [{'id': 102, 'name': 'Mouse', 'price': 500, 'quantity': 50}, {'id': 103, 'name': 'Keyboard', 'price': 1500, 'quantity': 30}, {'id': 101, 'name': 'Laptop', 'price': 50000, 'quantity': 10}]
PS C:\AIAC>
```

Task description #4: Smart Hospital Patient Management System

A hospital maintains records of thousands of patients with details such as patient ID, name, severity level, admission date, and bill amount. Doctors and staff need to:

1. Quickly search patient records using patient ID or name.
2. Sort patients based on severity level or bill amount for prioritization and billing.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms in terms of efficiency and suitability.
- Implement the recommended algorithms in Python.

The screenshot shows a VS Code editor with a file explorer on the left containing files like 1.2_ass.py, 8.2_ass.py, 9.3_ass.py, 12.1_ass.py, app.db, ass.py, ASS1.PY, Assignment.py, Lab exam.py, Mon.py, and wed.py. The main editor displays 12.1_ass.py with the following code:

```
79 # Sample patient records
80 patients = [
81     {"id": 201, "name": "Alice", "severity": 3, "admission_date": "2024-01-10", "bill": 5000},
82     {"id": 202, "name": "Bob", "severity": 5, "admission_date": "2024-01-12", "bill": 15000},
83     {"id": 203, "name": "Charlie", "severity": 2, "admission_date": "2024-01-11", "bill": 3000}
84 ]
85 # Search by ID using Dictionary (Fastest)
86 patient_dict = {patient["id"]: patient for patient in patients}
87 def search_by_id(patient_id):
88     """
89     Search for a patient by their ID using a dictionary for O(1) average time complexity.
90 """
```

The terminal at the bottom shows the execution of the script:

```
PS C:\AIAC> python 12.1_ass.py
Search ID 202: {'id': 202, 'name': 'Bob', 'severity': 5, 'admission_date': '2024-01-12', 'bill': 15000}
Sorted by Severity Level: [{'id': 203, 'name': 'Charlie', 'severity': 2, 'admission_date': '2024-01-11', 'bill': 3000}, {'id': 201, 'name': 'Alice', 'severity': 3, 'admission_date': '2024-01-10', 'bill': 5000}, {'id': 202, 'name': 'Bob', 'severity': 5, 'admission_date': '2024-01-12', 'bill': 15000}]
Sorted by Bill Amount: [{'id': 203, 'name': 'Charlie', 'severity': 2, 'admission_date': '2024-01-11', 'bill': 3000}, {'id': 201, 'name': 'Alice', 'severity': 3, 'admission_date': '2024-01-10', 'bill': 5000}, {'id': 202, 'name': 'Bob', 'severity': 5, 'admission_date': '2024-01-12', 'bill': 15000}]
PS C:\AIAC>
```

Task Description #5: University Examination Result Processing System

A university processes examination results for thousands of students containing roll number, name, subject, and marks. The system must:

1. Search student results using roll number.
2. Sort students based on marks to generate rank lists.

Student Task

- Identify efficient searching and sorting algorithms using AI assistance.
- Justify the choice of algorithms.
- Implement the algorithms in Python.

The screenshot shows a VS Code editor with a file explorer on the left containing files like 1.2_ass.py, 8.2_ass.py, 9.3_ass.py, 12.1_ass.py, app.db, ass.py, ASS1.PY, Assignment.py, Lab exam.py, Mon.py, and wed.py. The main editor displays 12.1_ass.py with the following code:

```
150 # Add sample test cases.
151 # Sample student records
152 students = [
153     {"roll_number": 301, "name": "David", "subject": "Math", "marks": 85},
154     {"roll_number": 302, "name": "Eve", "subject": "Science", "marks": 90},
155     {"roll_number": 303, "name": "Frank", "subject": "History", "marks": 80},
156 ]
157 # Search by Roll Number using Dictionary (Fastest)
158 student_dict = {student["roll_number"]: student for student in students}
159 def search_by_roll_number(roll_number):
160     """
161     Search for a student by their roll number using a dictionary for O(1) average time complexity.
162 """
```

The terminal at the bottom shows the execution of the script:

```
PS C:\AIAC> python 12.1_ass.py
Search Roll Number 302: {'roll_number': 302, 'name': 'Eve', 'subject': 'Science', 'marks': 90}
Sorted by Marks: [{'roll_number': 302, 'name': 'Eve', 'subject': 'Science', 'marks': 90}, {'roll_number': 301, 'name': 'David', 'subject': 'Math', 'marks': 85}, {'roll_number': 303, 'name': 'Frank', 'subject': 'History', 'marks': 80}]
PS C:\AIAC>
```

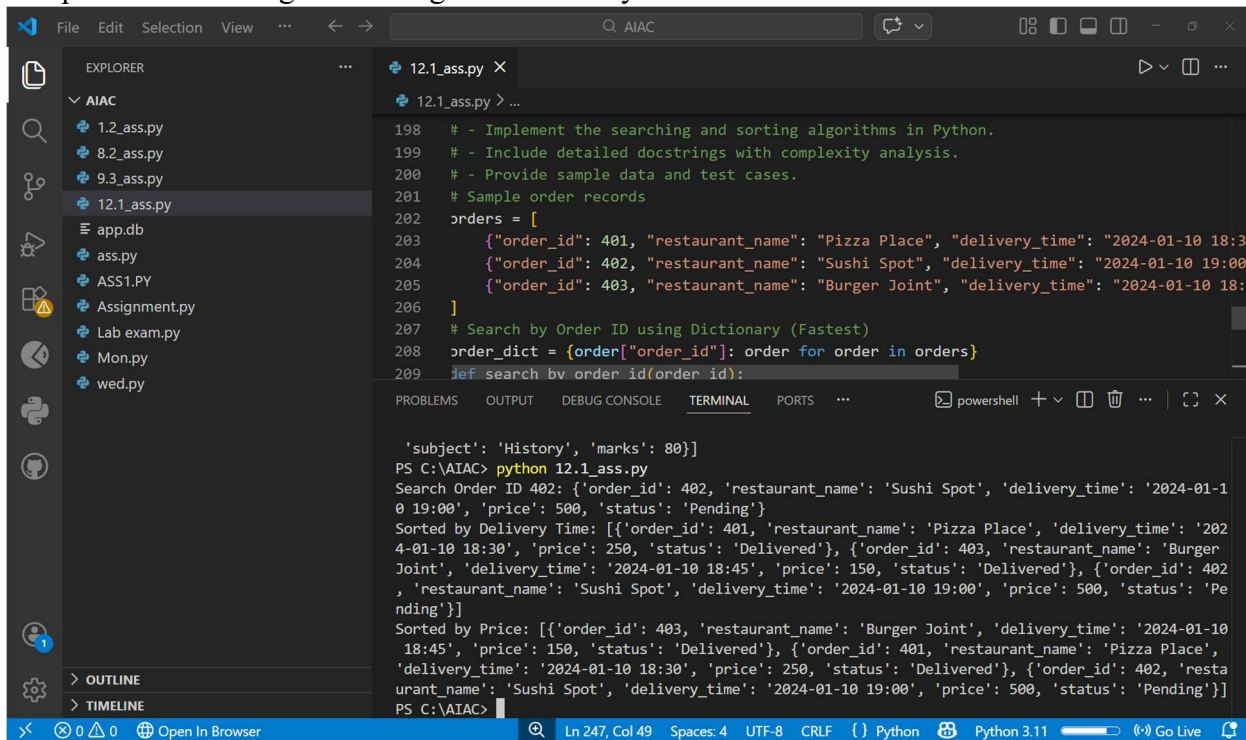
Task Description #6: Online Food Delivery Platform

An online food delivery application stores thousands of orders with order ID, restaurant name, delivery time, price, and order status. The platform needs to:

1. Quickly find an order using order ID.
2. Sort orders based on delivery time or price.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the algorithm selection.
- Implement searching and sorting modules in Python.



```
198 # - Implement the searching and sorting algorithms in Python.
199 # - Include detailed docstrings with complexity analysis.
200 # - Provide sample data and test cases.
201 # Sample order records
202 orders = [
203     {"order_id": 401, "restaurant_name": "Pizza Place", "delivery_time": "2024-01-10 18:30", "price": 250, "status": "Delivered"},
204     {"order_id": 402, "restaurant_name": "Sushi Spot", "delivery_time": "2024-01-10 19:00", "price": 500, "status": "Pending"},
205     {"order_id": 403, "restaurant_name": "Burger Joint", "delivery_time": "2024-01-10 18:45", "price": 150, "status": "Delivered"},
206 ]
207 # Search by Order ID using Dictionary (Fastest)
208 order_dict = {order["order_id"]: order for order in orders}
209 def search_by_order_id(order_id):
    """Search for an order by its ID using a dictionary.
    Returns the order details if found, otherwise returns None.
    Time Complexity: O(1)"""
    return order_dict.get(order_id)

if __name__ == "__main__":
    # Test the search function
    order = search_by_order_id(402)
    if order:
        print(f"Order found: {order}")
    else:
        print("Order not found")

    # Test the sorting function
    sorted_orders = sorted(orders, key=lambda order: order["delivery_time"])
    print("Sorted by Delivery Time:")
    for order in sorted_orders:
        print(f"Order ID: {order['order_id']}, Restaurant: {order['restaurant_name']}, Delivery Time: {order['delivery_time']}, Price: {order['price']}, Status: {order['status']}")

    sorted_orders = sorted(orders, key=lambda order: order["price"])
    print("Sorted by Price:")
    for order in sorted_orders:
        print(f"Order ID: {order['order_id']}, Restaurant: {order['restaurant_name']}, Delivery Time: {order['delivery_time']}, Price: {order['price']}, Status: {order['status']}")
```

```
PS C:\AIAC> python 12.1_ass.py
Search Order ID 402: {'order_id': 402, 'restaurant_name': 'Sushi Spot', 'delivery_time': '2024-01-10 19:00', 'price': 500, 'status': 'Pending'}
Sorted by Delivery Time: [{'order_id': 401, 'restaurant_name': 'Pizza Place', 'delivery_time': '2024-01-10 18:30', 'price': 250, 'status': 'Delivered'}, {'order_id': 403, 'restaurant_name': 'Burger Joint', 'delivery_time': '2024-01-10 18:45', 'price': 150, 'status': 'Delivered'}, {'order_id': 402, 'restaurant_name': 'Sushi Spot', 'delivery_time': '2024-01-10 19:00', 'price': 500, 'status': 'Pending'}]
Sorted by Price: [{'order_id': 403, 'restaurant_name': 'Burger Joint', 'delivery_time': '2024-01-10 18:45', 'price': 150, 'status': 'Delivered'}, {'order_id': 401, 'restaurant_name': 'Pizza Place', 'delivery_time': '2024-01-10 18:30', 'price': 250, 'status': 'Delivered'}, {'order_id': 402, 'restaurant_name': 'Sushi Spot', 'delivery_time': '2024-01-10 19:00', 'price': 500, 'status': 'Pending'}]
PS C:\AIAC>
```